Working with reference type Array :

Q1) WAP to work with reference array:

a) PARAM CONSTRUCTOR

```
package com.ravi.basic;

import java.util.Arrays;

class Customer
{
    private int id;
    private String name;
    private double bill;

    public Customer(int id, String name, double bill)
    {
        super();
        this.id = id;
        this.name = name;
        this.bill = bill;
    }

    @Override
    public String toString()
    {
        return "Customer [id=" + id + ", name=" + name + ", bill=" + bill + "]";
    }
}
public class ArrayDemo1 {

    public static void main(String[] args)
    {
        Customer customer[] = new Customer[5];
        customer[0] = new Customer(1, "Ajay", 1000);
        customer[1] = new Customer(2, "Sachin", 1500);
        customer[2] = new Customer(3, "John", 14000);
        customer[3] = new Customer(4, "Ajay", 1000);
        customer[4] = new Customer(5, "Sachin", 1500);


        for(Customer customer : customer)
        {
            System.out.println(customer);
        }
    }
}
```

b) Initialization through loop & search based on the id

```
package com.ravi.basic;

class Product
{
    private int id;
    private String name;
    private double price;

    public Product(int id, String name, double price) {
        super();
        this.id = id;
        this.name = name;
        this.price = price;
    }

    @Override
    public String toString() {
        return "Product [id=" + id + ", name=" + name + ", price=" + price + "]";
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}


public class ArrayDemo2 {

    public static void main(String[] args)
    {
        int size = Integer.parseInt(JO.readLn("Enter the size of Product object :"));
        Product []products = new Product[size];

        //initialize the array
        for(int i=0; i<products.length; i++)
        {
            int id = Integer.parseInt(JO.readLn("Enter the id of the product :"));
            String name = JO.readLn("Enter the name of the product :");
            double price = Double.parseDouble(JO.readLn("Enter the price of the
product :"));

            products[i] = new Product(id, name, price);
        }

        JO.println("Printing the Product Data :");

        for(Product product : products)
        {
            JO.println(product);
        }

        int searchId = Integer.parseInt(JO.readLn("Enter the id of the product to
search :"));

        boolean isFound = false;

        for(Product product : products)
        {
            if(product.getId()==searchId)
            {
                JO.println(product);
                isFound = true;
                break;
            }
        }

        if(!isFound)
        {
            JO.println("Product with "+searchId+" id is not available");
        }
    }

}
```

Q2) HEAP and STACK Diagram. [static member + array]

```
HEAP MEMORY
1000: AlphaObject
2000: AlphaObject -val 25
3000: AlphaObject -val 2
4000: AlphaArrayObject (9000, 2000)
                       5000
METHOD_AREA
100: ShowClass Loaded
200: AlphaClass Loaded, sval : 200, b 1000

STACK MEMORY
main_stack
ar1 : 2000
ar2 : 2000
ar : 4000
m : 6000

JO_stack
ar : 2000
ar : 2000
m : 6000

ar[0] = ar[1];
JO.println(ar[0].sval);//25
JO.println(ar[1].sval);//25

class Alpha
{
    int sval;

    static int sval = 200;
    static Beta b = new Beta();

    public Alpha(int val)
    {
    this.sval = val;
    }
}
public class Beta
{
    public static void main(String[] args)
    {
        Alpha ar1 = new Alpha(5);
        Alpha ar2 = new Alpha(2);

        Alpha [ar = {ar1,ar2};

    JO.println(ar);
    JO.println(ar[0].sval);
    JO.println(ar[1].sval);
    }

    public static Alpha[] fn(Alpha al, Alpha a2)
    {
        al.sval = 25;

    Alpha fa[] = new Alpha[2];

        return fa;
    }
}
```

Q3) WAP to modify the value of an array [Change Array Element]

```
modifyArrayElement

package com.ravi.basic;

import java.util.Arrays;

public class ArrayDemo7 {

    public static void main(String[] args)
    {
        int []value = {10,20,30,40,50};

        JO.println("Original Array is :"+Arrays.toString(value));

        int[] modifyArrayElement = modifyArrayElement(value);
        JO.println("Array Element after modification :"+Arrays.toString(modifyArrayElement));

        JO.println("Original Array after modification :"+Arrays.toString(value));
    }

    public static int[]  modifyArrayElement(int []arr)
    {
        arr[0] = 100;
        arr[1] = 200;
        return arr;
    }

}
```

Q4) How to sort Array elements in ascending order using nested for loop. [Bubble sort]

```
package com.ravi.basic;

import java.util.Arrays;

public class ArrayDemo8 {

    public static void main(String[] args)
    {
        int []arr = {3,5,7,1,4,2,7,1};

        JO.println("Original Array :"+Arrays.toString(arr));


        for(int i=0; i<arr.length; i++)
        {
            for(int j=i; j<arr.length; j++)
            {
                if(arr[i] > arr[j])
                {
                    //swapping logic
                    int temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }

        JO.println("Array elements in ascending order :"+Arrays.toString(arr));


    }
}
```

Q5) WAP to search an element in the existing array based on the index position.

```
package com.ravi.basic;

public class ArrayDemo9 {

    public static void main(String[] args)
    {
        int []arr = {1,2,3,4,5,6,7,2,3};
        int flag = 1;

        boolean isFound= false;

        for(int i=0; i<arr.length; i++)
        {
            if(flag==arr[i])
            {
                int index = i;
                JO.println("Flag - found at "+index+" index");
                isFound = true;
                break;
            }
        }

        if(!isFound)
        {
            JO.println("Flag - element is not available in the array");
        }
    }

}
```

Q6.1) Find the duplicate elements in the Array and print all the duplicate. - Assignement